

MODERN CLIS

Software Architecture

Ana Pérez Bango (UO294100), Martín Almoina Iglesias (UO300717), Andrés Álvarez Fernández (UO300170), Matias Valle Trapiella (UO300652)

Introduction

Despite the widespread implementation of graphical user interfaces (GUIs), Command Line Interfaces (CLIs) continue to play a fundamental role in modern software engineering. They have evolved with time into overall better and more refined tools compared to their beginnings as only performers of system-level operations.

The subject of the podcast is Derek Schafer, a senior software engineering leader with more than three decades of professional experience that has occupied several leadership positions throughout the years. Nevertheless, he insists that it is important to maintain a strong technical foundation and active involvement in engineering practices. He is also the author of a book called “CLI: A Practical Guide to Creating Modern Command Line Interfaces”, in which he addresses both the theoretical and practical aspects of CLI development, providing examples primarily in the Go programming language (selected due to its performance characteristics, simplicity, and suitability for building portable command line applications).

Definition and Historical Evolution of Command Line Interfaces

A Command Line Interface (CLI) is an interaction model that allows users to communicate with a computing system through a text-based terminal environment. It operates within the context of a shell, which interprets user input and executes commands or programs accordingly. This is not the same as individual commands, as those represent discrete operations while the CLI constitutes the broader interface that enables command execution, scripting, and system interaction (modern ones may invoke local system processes or communicate with remote services via APIs).

As for how it all began, the earliest forms of human-computer interaction relied on punch cards, which were later replaced by teletype machines. These devices allowed users to input commands and receive output sequentially, initially through printed output and subsequently via electronic terminals.

With the passing of time, the Unix operating system significantly advanced the development of CLIs by introducing powerful shells and a compositional design philosophy (exemplified by iconic Unix utilities such as `grep`, `cat`, and `sort`). This encouraged the creation of small, specialized programs that could be combined to perform complex tasks.

There was a significant decline in the perceived relevance of CLIs in the 1990s due to the proliferation of graphical user interfaces, which led them to be often regarded as obsolete and restricted to expert users. However, in more current times there has been a resurgence of CLIs due to the adoption of open-source software, cloud computing, and large-scale Distributed systems. This can be appreciated through the profound reliance of command line interaction to manage properly contemporary tools such as Git, Docker, and Kubernetes.

CLIs Implementation and Other Characteristics

Traditional CLIs often employed a flat command structure, where each operation was implemented as an independent command. In contrast, modern CLIs commonly adopt an object-oriented command model, in which a primary command is followed by subcommands, which improves scalability, maintainability, and usability. Modern CLIs are also designed to be self-documenting, reducing the need for external documentation. This approach enhances usability and supports automated systems, including large language models, that rely on structured command metadata.

CLIs have historically been implemented in a wide range of programming languages, including C, C++, Python, and shell scripting languages, with Go and Rust having gained prominence in recent years due to their advantageous features and characteristics.

Current CLIs also support both human-readable and machine-readable output formats, exemplified by text and tables for the prior one and JSON and CSV for the former. JSON is particularly important for interoperability with scripting environments and automation frameworks.

A significant challenge in the design of CLIs would be found in authentication, with the clear need to avoid insecure practices (hardcoding credentials, exposing secrets...). Recommended approaches for security measures include integration with external credential management systems and the use of short-lived authentication tokens.

In the future, it seems that CLIs will increasingly be designed for interaction with artificial intelligence systems, including large language models. They are also expected to place greater emphasis on compliance, auditing, and observability.

Conclusion

Command Line Interfaces have demonstrated remarkable adaptability over several decades of technological change. Through thoughtful design and adherence to established principles, modern CLIs continue to provide efficient, scalable, and reliable interaction mechanisms for complex software systems.

As articulated by Derek Schafer, effective CLI design requires careful consideration of usability, system architecture, security, and long-term maintainability. In the context of

cloud computing and AI-driven development, CLIs remain a critical component of professional software engineering practice.